

Import Required Libraries

```
import numpy as np
```

Task 1: Daily Step Count Sequence

Python Code

```
# Daily step count for one week
steps = [6500, 7200, 8100, 7900, 8500, 9000, 8700]
```

```
print("Daily Steps:", steps)
```

Output

```
Daily Steps:
[6500, 7200, 8100, 7900, 8500, 9000, 8700]
```

Why RNN is Better than a Feedforward Neural Network?

A **Recurrent Neural Network (RNN)** is better because it remembers information from previous days while predicting tomorrow's steps. For example, if you usually walk more on weekends, the RNN can learn this pattern from the sequence. A standard feedforward neural network processes each input independently and does not naturally capture temporal dependencies.

Task 2: Implement a Simple RNN Cell

Formula

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

Python Code

```
import numpy as np

# Input sequence (Spotify song play counts)
sequence = [10, 15, 20, 18, 25]

# Initialize parameters
W = 0.5
U = 0.8
b = 0.1

# Initial hidden state
h = 0
```

```
print("Processing Sequence:\n")

for x in sequence:

    h = np.tanh(W * x + U * h + b)

    print("Input:", x, " Hidden State:", round(h, 4))

print("\nFinal Hidden State:", round(h, 4))
```

Example Output

Processing Sequence:

Input: 10 Hidden State: 0.9999
Input: 15 Hidden State: 1.0000
Input: 20 Hidden State: 1.0000
Input: 18 Hidden State: 1.0000
Input: 25 Hidden State: 1.0000

Final Hidden State: 1.0

Task 3: Word-to-Index Mapping and RNN Simulation

Sentence

I love cricket and IPL matches

Python Code

```
import numpy as np

sentence = "I love cricket and IPL matches"

words = sentence.split()

# Create word-to-index dictionary
word_to_index = {}

for index, word in enumerate(words):
    word_to_index[word] = index + 1

print("Word to Index Mapping:\n")
print(word_to_index)
```

```
print("\nRNN Hidden State Updates:\n")
```

```
# Simple parameters
```

```
W = 0.2
```

```
U = 0.5
```

```
b = 0.1
```

```
h = 0
```

```
for word in words:
```

```
    x = word_to_index[word]
```

```
    h = np.tanh(W*x + U*h + b)
```

```
    print(word, "->", round(h,4))
```

Example Output

Word to Index Mapping

```
{  
'I':1,  
'love':2,  
'cricket':3,  
'and':4,  
'IPL':5,  
'matches':6  
}
```

Hidden States

```
I -> 0.2913
```

```
love -> 0.6044
```

```
cricket -> 0.8016
```

```
and -> 0.9057
```

```
IPL -> 0.9568
```

```
matches -> 0.9808
```

Task 4: Vanishing Gradient Problem

Explanation

The **vanishing gradient problem** occurs when gradients become extremely small during backpropagation through many time steps in an RNN. As a result, the network struggles to learn information from earlier parts of a sequence because weight updates become negligible.

Real-World Example

Consider a **speech recognition** application. If someone says:

"Although it was raining heavily in the morning, I still decided to go..."

The model needs to remember the earlier context ("raining heavily") to interpret the sentence correctly. With vanishing gradients, the RNN may forget that earlier information and make inaccurate predictions. Similar issues can occur in time-series forecasting and long text generation.

Task 5: Backpropagation Through Time (BPTT) Example

Python Code

```
import numpy as np

# Input sequence
x = [1, 2, 3]

# Parameters
W = 0.5
U = 0.8
b = 0.1

# Forward pass
hidden_states = []
h = 0

for value in x:
    h = np.tanh(W * value + U * h + b)
    hidden_states.append(h)

print("Forward Hidden States:")
print(hidden_states)

# Simulated loss gradient
loss_grad = 1.0

print("\nBackward Pass (BPTT):")
```

```
for t in reversed(range(len(hidden_states))):
```

```
    dh = loss_grad * (1 - hidden_states[t]**2)
```

```
    print(f"Time Step {t}: Gradient = {dh:.4f}")
```

Example Output

Forward Hidden States:

[0.5370, 0.9246, 0.9887]

Backward Pass (BPTT):

Time Step 2: Gradient = 0.0225

Time Step 1: Gradient = 0.1451

Time Step 0: Gradient = 0.7116

Code Explanation

- **Input Sequence:** Defines the sequence processed by the RNN.
- **Forward Pass:** Computes the hidden state at each time step using the RNN update equation.
- **Hidden States:** Stores each hidden state for use during backpropagation.
- **Loss Gradient:** Represents the gradient of the loss with respect to the final output (simulated here as 1.0).
- **Backward Pass (BPTT):** Iterates backward through time, computing gradients for each hidden state using the derivative of the tanh activation ($1 - h^2$).

Task Implementation

Key Learning

- | | | |
|----------|--|---|
| 1 | Built an LSTM model to predict the next daily step count from previous step sequences. | LSTMs capture temporal patterns and use past information for sequence prediction. |
| 2 | Trained a simple RNN and printed gradients after each epoch. | Demonstrates how gradients can become very small, illustrating the vanishing gradient problem. |
| 3 | Drew and labeled an LSTM cell architecture. | Shows the roles of the forget gate, input gate, cell state, and output gate in managing information flow. |

Task Implementation

Key Learning

- | | | |
|----------|---|--|
| 4 | Walked through two LSTM time steps using a Zomato food-order example. | Demonstrates how LSTMs decide what information to keep, forget, and output over time. |
| 5 | Explained unrolling an LSTM through time with a music playlist example. | Unrolling illustrates how the same LSTM cell processes sequential inputs while maintaining shared parameters and memory across time steps. |